

CVI Crash Course

Generated by Doxygen 1.16.1

1 Start Here	1
1.1 Introduction	1
1.1.1 About this Guide	1
1.1.2 Why Use CVI?	1
1.2 Getting Started	1
1.2.1 Open CVI	1
1.2.2 Familiarize Yourself	2
2 Configuration & Options	3
2.1 Build Settings	3
2.1.1 Build Configurations	3
2.1.2 Build Target Type	4
2.1.3 Build Target Settings	4
2.2 Environment	5
2.2.1 IncludePaths	5
2.3 Editor Preferences	5
3 Running a CVI Project	7
3.1 Running a CVI Project	7
3.1.1 Building a Project	7
3.1.2 Run Options	7
3.2 Debugging	7
3.2.1 Breakpoints & Watch Expressions	7
3.2.2 Step Commands	8
3.2.3 Debug Windows	8
3.2.4 Attach to Process	8
4 GUI Development	9
4.1 Getting Started	9
4.2 Working with the GUI Editor	9
4.2.1 GUI Editor Layout	9
4.2.2 Editing GUI Elements	10
5 Tips & Tricks	11
5.1 General Tips & Tricks	11
5.2 Hotkeys	11
6 Change Log	13
Index	15

Chapter 1

Start Here

1.1 Introduction

1.1.1 About this Guide

Welcome to the Arxtron CVI crash course. The aim of this guide is to succinctly summarize the key functionalities of CVI as used at Arxtron; however, it is not intended to be exhaustive. For further support, you can ask a colleague, or consult the official documentation.

If this is your first time reading the guide, it is recommended that you follow it sequentially and follow along in CVI when applicable.

1.1.2 Why Use CVI?

LabWindows/CVI is an IDE designed for test, measurement, and control applications. While fairly similar to any other IDE with C support, it features a handy GUI editor that can be used to quickly develop a frontend for any test setup. Additionally, it allows for easy integration with the suite of NI devices and programs that we use at Arxtron (primarily TestStand).

1.2 Getting Started

1.2.1 Open CVI

1. Press the Windows key and search for "CVI".
2. Open "NI LabWindows CVI 2019".

3. You will be met with the CVI Welcome menu. From here, you can access recent projects, start a new project from a template (or from scratch), and open help documents and example projects. Feel free to follow along in CVI by selecting any of the available options.



Figure 1.1 CVI Welcome Menu

1.2.2 Familiarize Yourself

Take a few moments to familiarize yourself with the general layout of the program.

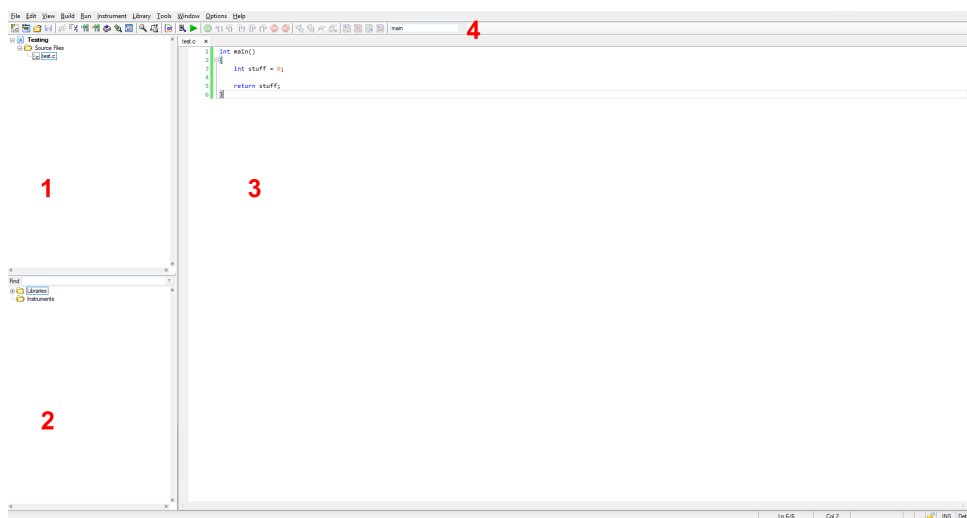


Figure 1.2 Default CVI Layout

The interface is divided into the following panes:

1. **Project Tree:** Used to browse the current project's structure.
2. **Library Tree:** Used to browse libraries.
3. **Window Confinement Region:** The primary region for editing code.
4. **Toolbar:** Contains icons which can be used to perform various useful actions.

If any of these panes do not appear by default, they can be toggled from within the **View** menu.

Additionally, CVI does not show line numbers by default. It is recommended that you select **View > Line Numbers** to toggle this setting.

Chapter 2

Configuration & Options

CVI contains a number of menus containing configuration settings and options. This section of the guide will briefly list the most important settings and their recommended states.

2.1 Build Settings

2.1.1 Build Configurations

Can be accessed via **Build > Configuration**. The resulting dropdown lists the available build configurations; more can be added by selecting **Manage Configurations**. The main configurations are:

- **Debug**: This configuration allows for debugging.
- **Release**: This configuration does not allow debugging, and is the most optimized for release.
- **Debug64**: 64-bit version of **Debug**.
- **Release64**: 64-bit version of **Release**.

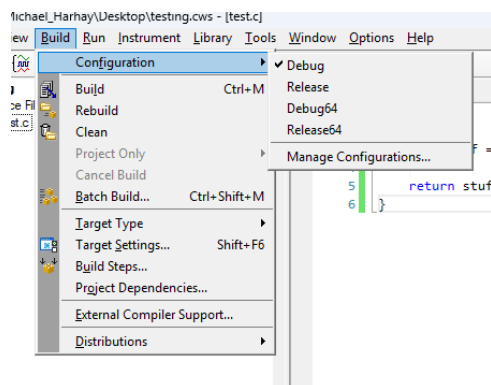


Figure 2.1 Build Configuration Menu

2.1.2 Build Target Type

Can be accessed via **Build > Target Type**. The resulting dropdown lists the available build target types:

- **Executable**: a standalone application that can be executed.
- **Dynamic Link Library (DLL)**: a library file that is linked at run time. DLLs are less portable and are not self-contained, but offer easier maintenance (simply replace the desired DLL without needing to recompile everything) and take up less space.
- **Static Library**: a library file that is linked at compile time. Static libraries have the advantage of being self-contained, however they are much larger. Additionally, any updates a static library require recompiling.

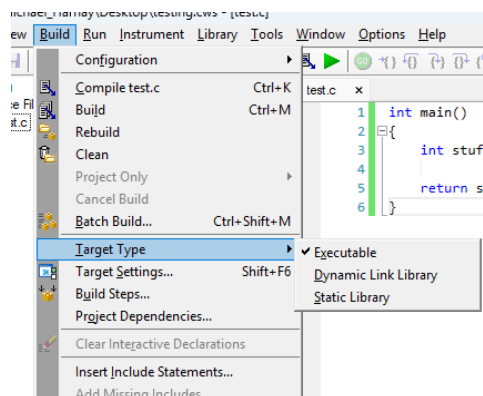


Figure 2.2 Build Target Type Menu

2.1.3 Build Target Settings

Can be accessed via **Build > Target Settings**. Target settings are used to configure the various target types. These settings typically aren't touched, but are good to know about.

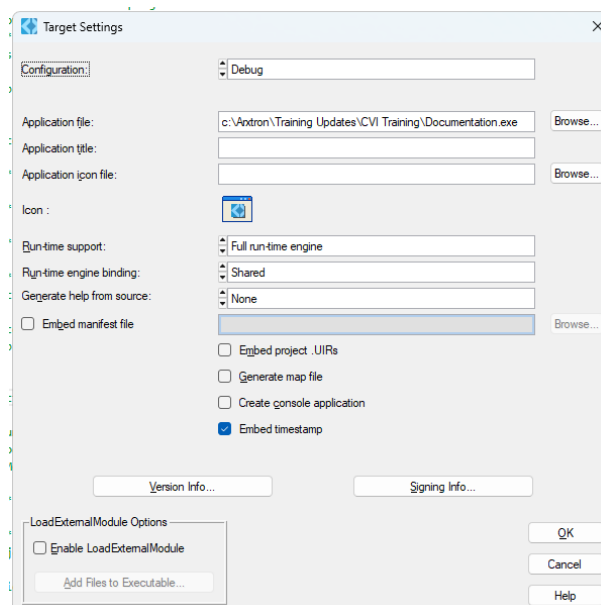


Figure 2.3 Build Target Settings Menu

2.2 Environment

2.2.1 IncludePaths

Can be accessed via **Options > Environment > Include Paths**. This menu determines CVI's search directories. Ensure that "C:\Arxtron\Libraries" is included.

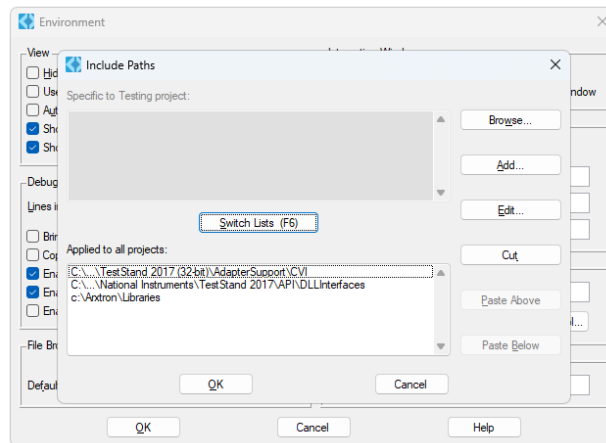


Figure 2.4 Include Paths Menu

2.3 Editor Preferences

Can be accessed via **Options > Editor Preferences**. Typically, these settings should not be changed, but are good to know about.

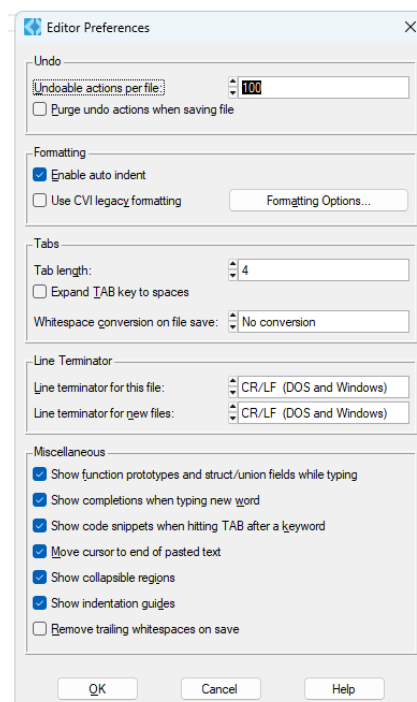


Figure 2.5 Editor Preferences Menu

Chapter 3

Running a CVI Project

3.1 Running a CVI Project

3.1.1 Building a Project

To compile a CVI project, follow these steps:

1. Ensure that the correct target type is selected.
2. If just the current file needs to be compiled, press **Ctrl+K** to do so. Otherwise, press **Ctrl+M** to build the entire project. Other build options can be found under **Build** (e.g. **Clean**, **Rebuild**).
3. Warnings, errors, and other build-related messages will appear in the **Build Output Pane**

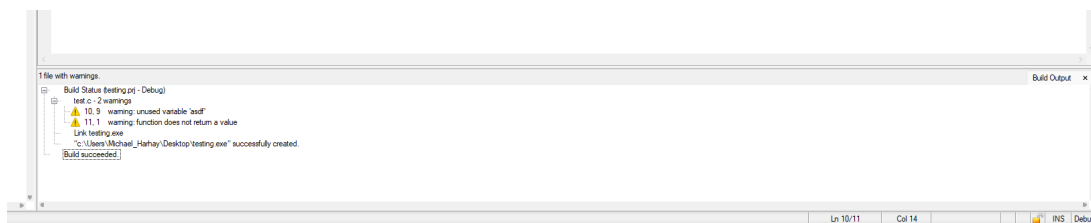


Figure 3.1 Build Output Pane

3.1.2 Run Options

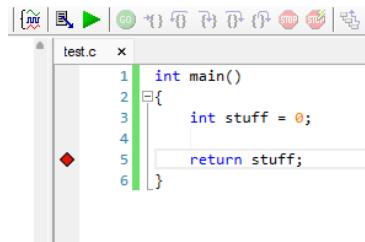
To run a compiled project, either run the executable or debug it in CVI (**Shift+F5**).

3.2 Debugging

3.2.1 Breakpoints & Watch Expressions

Breakpoints can be set by navigating to **Run > Breakpoints** (**Shift+F7**), and adding a breakpoint via the resulting menu.

Alternatively, clicking within the grey space directly to the left of a line number will enable a breakpoint.



Watch expressions can be set by selecting a line of code, then selecting **Run > Add Watch Expression** or by pressing **Shift+F9**.

3.2.2 Step Commands

While executing a sequence with breakpoints, standard debugging step commands can be used. They can be found under **Run**; here are some of the most useful ones:

- **Resume**: Resumes execution until the next breakpoint is reached (F5).
- **Set Next Statement**: Sets program to run from selected line (Ctrl+Shift+F7 / right click menu).
- **Step Over**: Steps over the current line without resuming execution (F8).
- **Step Into**: Steps into the current line of code (F10).

3.2.3 Debug Windows

CVI features a number of windows that are useful for debugging (found under **Window**). Here are a few useful ones to explore:

- **Memory**: displays the memory of the current program.
- **Resource Tracking**: displays both allocated and deallocated resources from the current program.
- **Variables and Call Stack**: displays all program variables and their values, as well as a standard call stack.
- **Watch**: displays all user-defined watch expressions.
- **Threads**: displays threads in the current program.

3.2.4 Attach to Process

When calling C code from a TestStand sequence or some other executable, you can open the corresponding source project in CVI and select **Run > Attach to Process** to attach to the process. This allows the current process to be debugged using CVI's debug features.

Similarly, **Run > Detach from Process** can be used to detach an attached CVI instance. This is often useful when stepping into CVI code from TestStand, as terminating the process from CVI without detaching will halt the entire TestStand sequence.

Chapter 4

GUI Development

4.1 Getting Started

CVI's built-in GUI editor is a handy way to create a simple frontend. To get started, either create a .uir file and add it to the project, or set up the [Arxtron Sequencer GUI](#).

4.2 Working with the GUI Editor

4.2.1 GUI Editor Layout

1. **GUI Editor Pane:** The region in which GUI editing takes place.
2. **Objects Pane:** This pane contains a list of present GUI objects.
3. **Object Properties Pane:** When a GUI object is selected, its properties appear in this pane.
4. **Help Pane:** This pane displays helpful messages about the current selection.

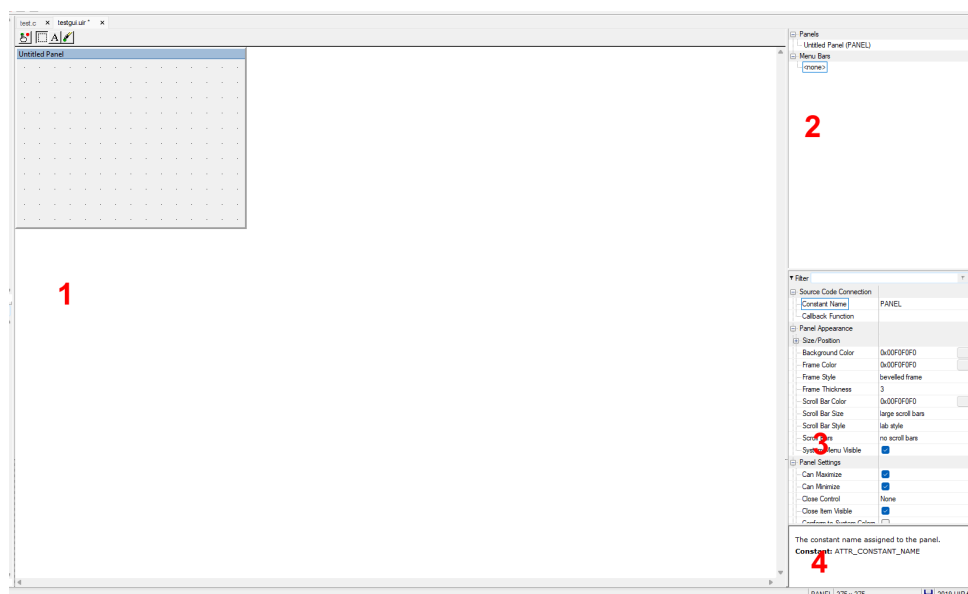


Figure 4.1 Default CVI GUI Editor Layout

4.2.2 Editing GUI Elements

To create a GUI element, right click anywhere and select the desired GUI element to add.

Once an element has been added, it can be edited by right clicking and selecting **Edit Control**. Alternatively, the Object Properties Pane can be used for editing.

To learn more about a GUI element, right click it and select **Control Help**.

Chapter 5

Tips & Tricks

The following tips and tricks are helpful to know.

5.1 General Tips & Tricks

- **Help Menu:** Press **F1** to access the CVI help documentation.
- **Generate Prototype:** Select **Build > Generate Prototypes** to generate a file containing declarations for global and static functions, as well as external declarations for global variables. These can be copied into your existing source and header files.
- **Tags:** Select a line and press **Shift+F2** to "tag" or "bookmark" that line. Then, press **F2** to cycle through all tagged lines.

5.2 Hotkeys

- **Comment Selection:** Select code to comment out, press **Ctrl+F3**
- **Uncomment Selection:** Select code to uncomment, press **Ctrl+F4**
- **Show Function Prototype:** Place cursor in function, press **Ctrl+Shift+Space**
- **Find:** **Ctrl+F**
- **Replace:** **Ctrl+H**
- **Go to line:** **Ctrl+L**
- **Build project:** **Ctrl+M**
- **Build specific file within project:** **Ctrl+K**
- **Go to line:** Select item to check definition of, press **Ctrl+I**
- **Go back:** After going to definition via **Ctrl+I**, press **Ctrl+G** to go back

Chapter 6

Change Log

Version	Author	Date	Changes
1.↔ 0.0	Michael Harhay	26/3/2026	Initial Release

Index

Change Log, [13](#)
Configuration & Options, [3](#)
GUI Development, [9](#)
Running a CVI Project, [7](#)
Start Here, [1](#)
Tips & Tricks, [11](#)